Seminar Report

On

MOVIE RECOMMENDATION SYSTEM

Submitted by

Aashish Bedi – Registration No.: 12209606

Bachelor of Technology

IN

(Computer Science and Engineering)

Under the Supervision of

_____

(Manish Kumar Sharma)

LOVELY PROFESSIONAL UNIVERSITY, PUNJAB

(Jan – Mar 2026)

DECLARATION

I hereby declare that the seminar report titled “Movie Recommendation System” submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a record of my own work carried out during the academic session 2025–26.

I further declare that this report has not been submitted, either in part or in full, to any other institution or university for the award of any degree or diploma.

I confirm that the content of this report is original and prepared by me. Any references used have been duly acknowledged. I also declare that the use of Artificial Intelligence (AI) tools, if any, has been minimal and the AI-generated content in this report is less than 10%, ensuring that most of the work reflects my own understanding and effort.

I take full responsibility for the authenticity and originality of the work presented in this report.

Name of the Student: Aashish Bedi
Registration Number: 12209606
Course: B.Tech in Computer Science and Engineering

Signature of the Student: _____Aashish Bedi_____

Date: March 25, 2026

ABSTRACT

The Movie Recommendation System is a content-based filtering application built using Natural Language Processing (NLP) and machine learning techniques. The system processes metadata from a dataset of over 45,000 movies to identify similarities between films and generate personalized recommendations. Using TF-IDF (Term Frequency–Inverse Document Frequency) vectorization and cosine similarity, the application transforms movie attributes such as genres, keywords, cast, crew, and plot overviews into a high-dimensional vector space to compute pairwise movie similarity.

The system is deployed as an interactive web application using Streamlit and integrates with The Movie Database (TMDB) API to display real-time movie posters alongside recommendations. Testing demonstrates strong performance on franchise movies, director-specific catalogues, and genre-based recommendations, with results such as correctly recommending Christopher Nolan films for “The Dark Knight” input and Pixar films for “Toy Story” input.

This report documents the design, implementation, results, and future scope of the system, covering the underlying algorithms, system architecture, dataset analysis, comparative analysis, and research references.

Live Application: https://movierecommendation-4bn6n5byk3akrxdjtsrssr.streamlit.app/

Chapter-1: Introduction

1.1 Title of the Seminar Topic

Movie Recommendation System Using Natural Language Processing and Machine Learning

1.2 Background and Importance of the Topic

In the digital age, the abundance of content available on streaming platforms and entertainment websites has created a significant challenge for users: information overload. With thousands of movies released every year and massive libraries available on platforms like Netflix, Amazon Prime, and Disney+, users often spend considerable time searching for content that matches their preferences.

Movie recommendation systems have emerged as a critical solution to this problem. These systems leverage machine learning algorithms and data mining techniques to analyse user behaviour, preferences, and movie characteristics to provide personalised recommendations [1]. By predicting what movies a user might enjoy based on past interactions and similar users' behaviours, recommendation systems enhance user experience, increase engagement, and reduce the time spent searching for content.

The importance of recommendation systems extends beyond user convenience. For businesses, these systems drive customer retention, increase content consumption, and provide valuable insights into user preferences. Studies show that recommendation systems can significantly improve user satisfaction and platform engagement, making them an essential component of modern entertainment platforms [2].

1.3 Objectives of the Seminar

1. To understand the fundamental concepts and algorithms behind movie recommendation systems.
2. To explore different approaches including content-based filtering and collaborative filtering.
3. To implement a functional movie recommendation system using machine learning techniques.
4. To analyse and compare the effectiveness of different recommendation algorithms.

5. To evaluate the system's performance in providing accurate and relevant movie suggestions.
6. To identify potential improvements and future research directions in recommendation systems.

1.4 Brief Overview of the Approach or Methodology

This project implements a movie recommendation system using a content-based filtering approach. The system analyses movie attributes such as genres, cast, crew, keywords, and plot descriptions to identify similarities between movies. By computing similarity scores based on these features, the system recommends movies that share similar characteristics with the user's selected title.

The implementation utilises Python with popular machine learning libraries including Pandas for data manipulation, Scikit-learn for feature extraction and similarity computation, and NumPy for numerical operations. The system employs TF-IDF vectorisation and cosine similarity to measure resemblance between movies based on content features [3]. The final application is deployed on Streamlit Cloud with TMDB API integration for real-time poster fetching.

Chapter-2: Literature Review

2.1 Overview of Recommendation Systems

Recommendation systems are information filtering systems that predict user preferences and suggest items accordingly. They have become ubiquitous in e-commerce, entertainment, social media, and various other domains. The fundamental goal is to provide personalised experiences by matching users with relevant content [4].

2.2 Types of Recommendation Systems

2.2.1 Collaborative Filtering

Collaborative filtering makes recommendations based on user–user similarity or item–item similarity. This approach assumes that users who agreed in the past will agree in the future. There are two main variants [5]:

- User-based Collaborative Filtering: Identifies users with similar preferences and recommends items liked by similar users.
- Item-based Collaborative Filtering: Recommends items similar to those the user has previously liked, based on ratings from all users.

Advantages: Does not require item metadata; can discover unexpected patterns.
Limitations: Cold-start problem for new users/items; scalability issues with large datasets.

2.2.2 Content-Based Filtering

Content-based filtering recommends items based on the features of items and user preferences. The system creates a profile for each user based on the characteristics of items they have previously interacted with [6]. For movies, features include genres, actors, directors, keywords, and plot summaries.

Advantages: No cold-start problem for new users; can recommend niche items; provides explainable recommendations.
Limitations: Limited diversity in recommendations; requires detailed item metadata.

2.2.3 Hybrid Approaches

Hybrid recommendation systems combine collaborative and content-based filtering to leverage

the strengths of both approaches while mitigating their individual weaknesses. These systems often achieve superior performance compared to single-method approaches [7].

2.3 Machine Learning Algorithms in Recommendation Systems

Various machine learning algorithms are employed in recommendation systems:

- K-Nearest Neighbours (KNN): Identifies similar items or users based on distance metrics.
- Matrix Factorisation: Decomposes the user-item interaction matrix into lower-dimensional representations.
- Neural Collaborative Filtering: Uses deep learning to model complex user-item interactions.
- Natural Language Processing: Analyses textual features like plot descriptions and reviews.
- Clustering Algorithms: Groups similar items or users for efficient recommendation generation.

2.4 Recent Advances

Recent research has focused on incorporating deep learning techniques, particularly neural networks and autoencoders, to capture non-linear relationships in user-item interactions [8]. Recurrent Neural Networks (RNNs) have been applied to model sequential user behaviour, while attention mechanisms help identify important features for personalised recommendations.

Studies have shown that content-based approaches using advanced feature extraction techniques can achieve precision improvements of 20–25% compared to traditional methods, particularly for new or niche movies with limited user ratings [9]. This supports the choice of content-based filtering as the core methodology for this project.

Chapter-3: Conceptual Study and Seminar Work

3.1 System Architecture

The movie recommendation system follows a multi-stage pipeline architecture consisting of six interconnected modules:

7. Data Collection Module: Gathers movie information from datasets including metadata, ratings, and user interactions.
8. Data Preprocessing Module: Cleans, transforms, and prepares data for feature extraction.
9. Feature Engineering Module: Extracts and combines relevant movie features into a unified representation.
10. Similarity Computation Module: Calculates cosine similarity scores between all movie pairs.
11. Recommendation Generation Module: Produces a ranked list of the top-N most similar movies.
12. User Interface Module: Provides a Streamlit-based interface for user interaction and displays recommendations with TMDB posters.

3.2 Core Concepts and Algorithms

3.2.1 Content-Based Filtering Implementation

The content-based approach analyses movie attributes to identify similarities. The key steps are as follows:

Step 1 – Feature Selection: The system considers multiple movie features: Genres (Action, Comedy, Drama, Thriller), Cast (main actors), Crew (directors, key personnel), Keywords (descriptive thematic tags), and Overview (plot summary).

Step 2 – Feature Combination: All selected features are combined into a single text representation ('soup' string) for each movie, creating a comprehensive content profile.

Step 3 – Text Vectorisation (TF-IDF): TF-IDF is used to convert textual features into numerical vectors [10]. It assigns weights to terms based on Term Frequency (how often the term appears in the document) and Inverse Document Frequency (the log of the ratio of total documents to documents containing the term). This emphasises unique characteristics while reducing the impact of common terms.

Formally: $TF\text{-}IDF(t, d) = TF(t, d) \times \log(N / df(t))$, where N is the total number of documents and df(t) is the document frequency of term t.

Step 4 – Cosine Similarity Computation: Cosine similarity measures the angle between feature vectors: $\cos(\theta) = (A \cdot B) / (\|A\| \|B\|)$. The score ranges from 0 (no similarity) to 1 (identical). Cosine similarity is preferred over Euclidean distance in high-dimensional text spaces because it is invariant to vector magnitude.

3.2.2 Recommendation Generation Algorithm

13. Accept a movie title as input from the user.
14. Retrieve the TF-IDF feature vector for the input movie.
15. Calculate cosine similarity scores between the input movie and all other movies.
16. Sort movies by similarity score in descending order.
17. Return the top-5 movies, excluding the input movie itself.
18. Display recommendations with movie posters fetched from the TMDB API.

3.3 Tools and Technologies

Category	Technology	Purpose
Programming Language	Python 3.x	Core implementation
Data Manipulation	Pandas	Dataset handling and preprocessing
Machine Learning	Scikit-learn	TF-IDF vectorisation and similarity
Numerical Computing	NumPy	Mathematical operations
NLP	NLTK	Stemming and text preprocessing
Web Application	Streamlit	UI and cloud deployment
External API	TMDB API v3	Real-time movie poster fetching
Development Environment	Jupyter Notebook	Exploratory development and analysis
Version Control	Git / GitHub	Source code management

Table 3.1: Technologies and Tools Used

3.4 Dataset Description

The system uses the TMDB movies metadata CSV containing the following fields per record:

- Movie ID: Unique identifier for each movie.
- Title: Name of the movie.
- Genres: Categories associated with the movie (stored as JSON strings, parsed at preprocessing).
- Cast: List of main actors appearing in the movie.
- Crew: Directors, producers, and key crew members.
- Keywords: Descriptive thematic tags.
- Overview: Plot summary and description.
- Release Date: Year of release.
- Ratings: User ratings and vote counts.

3.5 Implementation Workflow

19. Data Loading: Import movie dataset using Pandas from movies_metadata.csv.
20. Data Cleaning: Handle missing values, remove duplicates, and standardise formats.
21. Feature Extraction: Parse JSON-encoded fields containing cast, crew, and keywords into plain text.
22. Feature Engineering: Combine selected features into a unified 'soup' text representation per movie.
23. Vectorisation: Apply TF-IDF transformation (5,000 feature cap) to create feature vectors.
24. Similarity Matrix: Compute pairwise cosine similarity for all movies and serialise to pickle files (tfidf.pkl, tfidf_matrix.pkl, indices.pkl).
25. Recommendation Function: Implement a function that retrieves the top-5 similar movies for any input title.
26. Deployment: Launch Streamlit application on Streamlit Cloud with TMDB API integration for poster display.

Chapter-4: Results and Discussion

4.1 System Performance

4.1.1 Sample Recommendations

The implemented movie recommendation system successfully generates relevant recommendations based on movie content features. Testing across popular titles confirms strong performance in identifying movies with similar characteristics:

Input Movie: "The Dark Knight"
Recommended movies: Batman Begins, The Dark Knight Rises, Inception, The Prestige, Man of Steel.
Analysis: The recommendations correctly identify movies with similar genres (action, thriller), shared cast members (Christian Bale), and the same director (Christopher Nolan). This demonstrates the system's ability to capture multiple dimensions of similarity simultaneously.

Input Movie: "Toy Story"
Recommended movies: Toy Story 2, Toy Story 3, Finding Nemo, Monsters Inc., The Incredibles.
Analysis: The system successfully identifies animated movies from Pixar Studios, demonstrating its capability to recognise studio affiliation and animation genre as strong similarity indicators.

4.2 Advantages of the Content-Based Approach

27. No Cold Start for New Users: The system can recommend movies to new users immediately based on a single movie preference, without requiring historical user data.
28. Transparency: Recommendations are explainable based on specific movie attributes, making the system interpretable and auditable.
29. Independence: Does not require data from other users, ensuring privacy and avoiding collaborative filtering biases.
30. Niche Recommendations: Can recommend less popular movies with similar content characteristics that collaborative systems might miss.
31. Immediate Updates: New movies can be recommended as soon as their metadata is added, without waiting for rating accumulation.

4.3 Limitations and Challenges

32. Limited Diversity: The system tends to recommend movies very similar to the input, potentially creating a “filter bubble” effect.
33. Serendipity Problem: Difficulty in recommending unexpected but potentially interesting movies from different genres.
34. Feature Dependency: Recommendation quality heavily depends on the comprehensiveness and accuracy of movie metadata.
35. New Genre Problem: Cannot effectively recommend movies if the user’s preferred genre is not well-represented in features.
36. Overspecialisation: May recommend too many movies from the same franchise or director, reducing variety.

4.4 Comparative Analysis

Aspect	Content-Based	Collaborative
Cold Start Problem	Low impact	High impact
Data Requirements	Item features	User interactions
Scalability	High	Moderate
Diversity	Lower	Higher
Explainability	High	Low
New Item Handling	Immediate	Requires ratings

Table 4.1: Comparison of Content-Based and Collaborative Filtering Approaches

4.5 Key Observations

- TF-IDF vectorisation effectively captures distinctive movie characteristics while minimising noise from common terms.
- Cosine similarity provides robust measurement of content similarity across high-dimensional feature spaces.
- Combining multiple features (genre, cast, crew, keywords) produces more accurate recommendations than single-feature approaches.
- The system performs exceptionally well for franchise movies and works by specific directors.

- Genre-based similarity often dominates recommendations, suggesting potential for weighted feature importance in future versions.

4.6 Discussion on Improvements

Several enhancements could improve system performance and address the identified limitations:

37. Hybrid Approach: Combining content-based filtering with collaborative filtering to leverage user behaviour data alongside item features.
38. Weighted Features: Implementing feature importance weights to balance genre, cast, and crew contributions dynamically.
39. User Profiles: Creating user preference profiles based on multiple liked movies for improved personalisation.
40. Temporal Factors: Incorporating movie release dates and trending patterns to surface fresher content.
41. Sentiment Analysis: Analysing user reviews to enhance content understanding beyond structured metadata.
42. Deep Learning: Employing neural networks (autoencoders, attention mechanisms) to capture complex feature interactions.

Chapter-5: Conclusion and Future Scope

5.1 Summary of the Seminar Work

This seminar work successfully designed and implemented a movie recommendation system using content-based filtering and machine learning techniques. The system analyses movie attributes including genres, cast, crew, and keywords to identify similar movies and generate personalised recommendations. The implementation utilised Python with Pandas, Scikit-learn, and NumPy libraries to create a fully functional recommendation engine, deployed as an interactive Streamlit web application with real-time TMDB API integration.

5.2 Major Learning Outcomes

- Understanding of recommendation system architectures and algorithms.
- Practical experience with content-based filtering implementation.
- Knowledge of feature engineering and text vectorisation techniques.
- Hands-on application of machine learning concepts to real-world problems.
- Data preprocessing and cleaning skills for large movie datasets.
- Proficiency in Python libraries for data science and machine learning.
- Insight into the advantages and limitations of different recommendation approaches.
- Understanding of similarity metrics and their applications in high-dimensional spaces.

5.3 Conclusions Drawn from the Study

43. Content-based filtering provides an effective approach for movie recommendations when rich metadata is available.
44. The combination of multiple content features (genres, cast, crew, keywords) significantly improves recommendation quality compared to single-feature approaches.
45. TF-IDF vectorisation and cosine similarity form a robust foundation for content-based recommendation systems.
46. The system demonstrates particular strength in recommending franchise movies, sequels, and works by specific directors or studios.
47. While content-based systems offer advantages in explainability and immediate applicability, they face challenges in providing diverse recommendations.
48. The success of recommendation systems depends heavily on the quality, completeness, and accuracy of input data.

5.4 Future Scope and Enhancements

5.4.1 Hybrid Recommendation System

Integrating collaborative filtering with the existing content-based approach would create a hybrid system that leverages both user behaviour patterns and movie content features. This would address the overspecialisation limitation while maintaining the strengths of content-based filtering [11].

5.4.2 Deep Learning Integration

Implementing neural network architectures such as Autoencoders for learning compressed movie representations, Recurrent Neural Networks for capturing sequential user preferences, Attention Mechanisms for identifying important features dynamically, and Neural Collaborative Filtering for modelling complex user-item interactions.

5.4.3 Personalised User Profiles

Developing comprehensive user profiles based on multiple movie preferences and ratings, viewing history and watch patterns, temporal preferences (mood-based recommendations), and explicit feedback and implicit signals.

5.4.4 Advanced Feature Engineering

Enhancing the feature set with sentiment analysis of user reviews, plot similarity using advanced NLP techniques (e.g., BERT embeddings), visual features from movie posters and trailers, audio features from soundtracks, and award nominations and critical acclaim data.

5.4.5 Real-Time Recommendation System

Developing a full web application or API that provides real-time recommendation generation, an interactive user interface with feedback mechanisms for continuous model improvement, integration with major streaming platforms, and social features for sharing recommendations.

5.4.6 Diversity, Serendipity, and Scalability

Future work should also address diversity and serendipity by implementing algorithms that balance accuracy with discovery, introduce controlled randomness, and provide genre exploration suggestions. Scalability improvements include distributed computing for similarity calculations, approximate nearest neighbour algorithms (e.g., FAISS) for faster retrieval, and caching mechanisms for popular queries.

5.5 Broader Impact

Movie recommendation systems represent a critical component of modern entertainment platforms. As content libraries continue to expand, intelligent recommendation systems become increasingly essential for user engagement and satisfaction. The techniques and concepts explored in this seminar work have applications beyond movies, extending to music, books, products, news articles, and various other domains.

The future of recommendation systems lies in creating more intelligent, context-aware, and personalised experiences that balance accuracy with diversity, privacy with personalisation, and efficiency with effectiveness.

5.6 Professional Profile and Repository Details

GitHub Repository: https://github.com/AashishBedi/Movie_Recommendation
Live Application: https://movierecommendation-4bn6n5byk3akrxdjtsrssr.streamlit.app/
LinkedIn Profile: https://www.linkedin.com/in/aashishbedi/
GitHub Profile: https://github.com/AashishBedi

REFERENCES

[1] Resnick, P., & Varian, H. R. (1997). Recommender systems. Communications of the ACM, 40(3), 56–58.

[2] Jayalakshmi, S., Ganesh, N., Čep, R., & Senthil Murugan, J. (2022). Movie recommender systems: Concepts, methods, challenges, and future directions. Sensors, 22(13), 4904.

[3] Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. Information Processing & Management, 24(5), 513–523.

[4] Ricci, F., Rokach, L., & Shapira, B. (2015). Recommender Systems Handbook (2nd ed.). Springer.

[5] Sarwar, B., Karypis, G., Konstan, J., & Riedl, J. (2001). Item-based collaborative filtering recommendation algorithms. Proceedings of the 10th International Conference on World Wide Web, 285–295.

[6] Pazzani, M. J., & Billsus, D. (2007). Content-based recommendation systems. In The Adaptive Web (pp. 325–341). Springer.

[7] Burke, R. (2002). Hybrid recommender systems: Survey and experiments. User Modeling and User-Adapted Interaction, 12(4), 331–370.

[8] Zhang, S., Yao, L., Sun, A., & Tay, Y. (2019). Deep learning based recommender system: A survey and new perspectives. ACM Computing Surveys, 52(1), 1–38.

[9] Rakesh, S., Kumar, M. N., & Reddy, P. V. (2024). Movie recommendation system using content based filtering. Al-Bahir Journal for Engineering and Pure Sciences, 4(1).

[10] Ramos, J. (2003). Using TF-IDF to determine word relevance in document queries. Proceedings of the First Instructional Conference on Machine Learning, 133–142.

[11] Adomavicius, G., & Tuzhilin, A. (2005). Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. IEEE Transactions on Knowledge and Data Engineering, 17(6), 734–749

--- ---